Thadomal Shahani Engineering College, Bandra

Name: Yash Singh	Subject: ASD&D
T.E. Batch: T22 Sem: 5th Roll No: 2401118	Date: 2nd September 2026

Department of Artificial Intelligence & Data Science

Experiment No. 2

Containerisation using Docker

Aim

To containerise an application using Docker.

Learning Outcome (LO4)

Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory

Containerisation

Docker is an open-source containerisation platform that enables applications to run in isolated environments called containers. A container packages the application together with all its required libraries, dependencies and configuration files, ensuring consistent execution across different systems. Because containers share the host operating system's kernel rather than virtualising an entire guest OS, they are lightweight, portable, and start up much faster than traditional virtual machines.

Docker Images and Containers

A Docker image is a read-only template that contains everything needed to create a container — the base operating system files, installed software packages, and application files. Images are stored in repositories such as Docker Hub and can be downloaded locally using the docker pull command. Once an image is available locally, it can be used to instantiate one or more running containers. A container is simply a running (or stopped) instance of an image, with its own writable filesystem layer on top of the image's read-only layers.

Common Docker Commands

- **docker --version:** verifies the installed Docker version.
- **docker search <name>:** searches Docker Hub for images matching a name.
- **docker pull <image>:** downloads the specified image from a registry to the local machine.
- **docker run -it <image>:** creates and starts a new container from an image, attaching an interactive terminal (-it) so commands can be run inside it directly.
- **exit:** leaves the container's shell, which by default also stops the container.

Once inside a running container's shell, ordinary Linux commands such as mkdir, cd, touch, echo and cat can be used to create directories and files and to perform basic file operations, exactly as on a native Linux system.

Implementation

Step 1: Open Terminal

Opened Windows PowerShell to run Docker commands from the command line.

Step 2: Check Docker Version

Command: `docker --version`



Fig. 1: PowerShell session showing docker --version — Docker version 28.4.0, build d8eb465

Step 3: View Docker Image Management Commands

Command: `docker image`

Running docker image without a subcommand displays the usage and the full list of image-management subcommands available (build, history, import, inspect, load, ls, prune, pull, push, rm, save, tag), confirming that the Docker image-management tooling is installed and accessible.

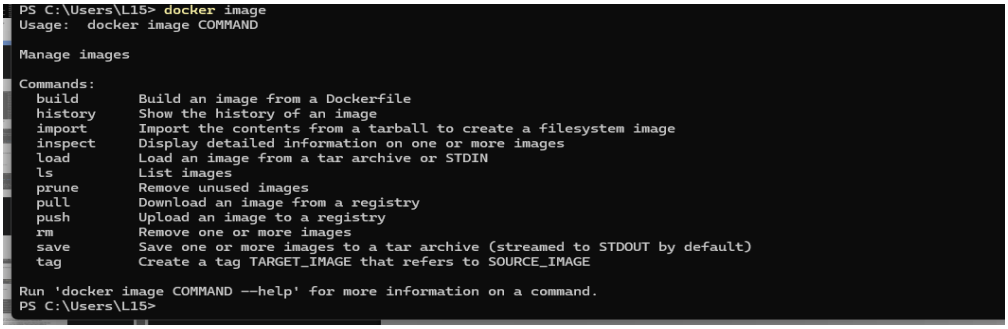


Fig. 2: Output of docker image, listing the available image-management subcommands

Step 4: Search Ubuntu Image

Command: `docker search ubuntu`

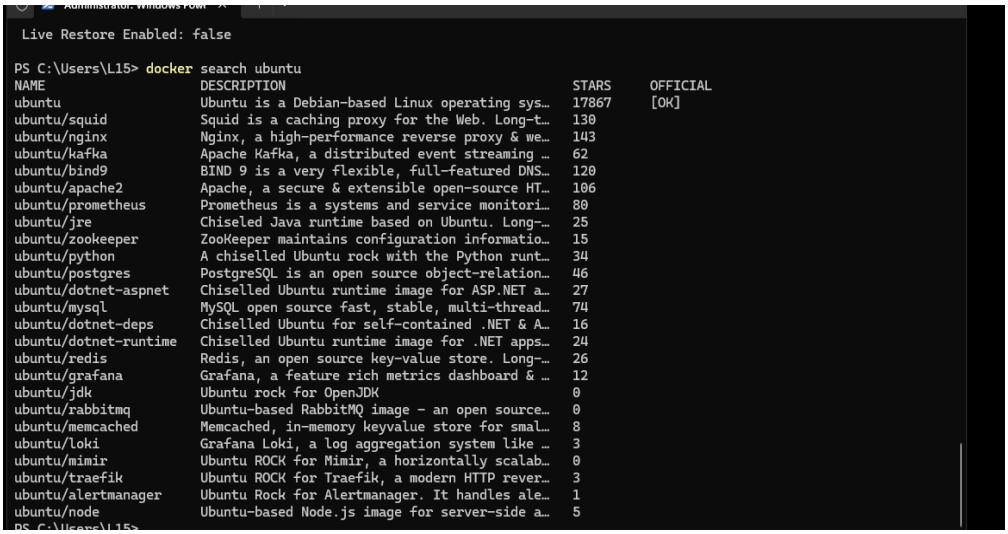


Fig. 3: Results of docker search ubuntu, listing Ubuntu-related images available on Docker Hub with their star counts

Step 5: Pull Ubuntu Image

Command: `docker pull ubuntu`

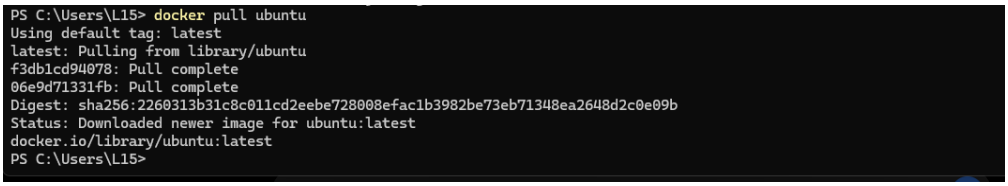


Fig. 4: docker pull ubuntu downloading the ubuntu:latest image layers and completing successfully

Step 6: Run Container Interactively

Command: `docker run -it ubuntu`

This creates a new container from the ubuntu image and attaches an interactive shell, dropping into a root prompt inside the container.

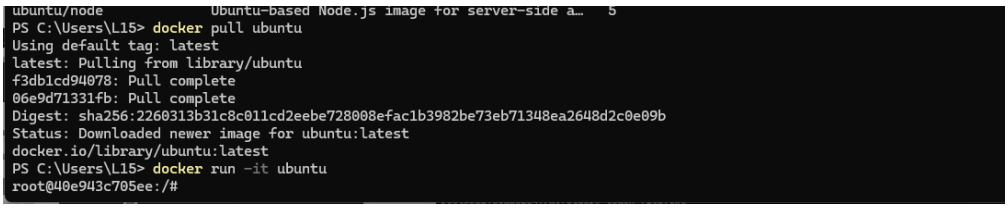


Fig. 5: docker run -it ubuntu starting the container and opening a root shell inside it

Step 7: Execute Linux Commands Inside the Container

Commands:

```
mkdir myfolder
cd myfolder
touch file.txt
echo "hello" >> file.txt
cat file.txt
```

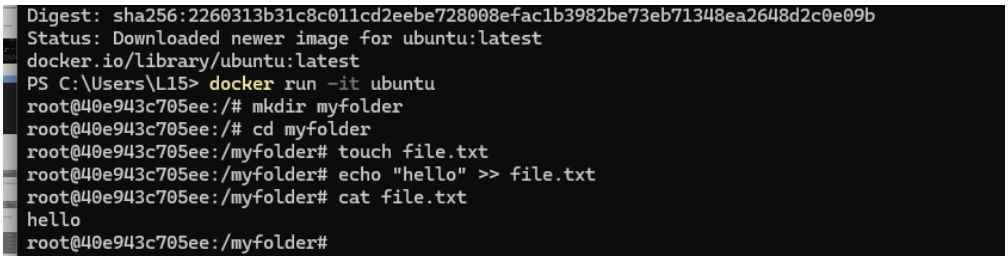


Fig. 6: Creating a folder and file inside the running container, writing to it, and reading it back with cat (output: hello)

Step 8: Exit Container

Command: `exit`



Fig. 7: Exiting the container's shell and returning to the host PowerShell prompt

Conclusion

In this experiment, the fundamentals of Docker were learned by performing basic image and container operations. The Docker installation was verified using docker --version, the available image-management subcommands were explored using docker image, and the Ubuntu image was searched for and downloaded from Docker Hub using docker search and docker pull. A container was then created from the Ubuntu image using docker run -it, and basic Linux commands (mkdir, cd, touch, echo and cat) were executed inside it to create and read back a file, before exiting the container's shell. This experiment helped in understanding Docker images, containers, and containerisation, and how Docker provides an efficient, isolated and portable environment for running applications independent of the underlying host configuration.